

Project Report: Automated Podcast Transcription and Topic Segmentation

Project Title: Automated Podcast Transcription and Topic Segmentation

Dataset Used: Audio_recording_whisper

Technology Stack: Python, OpenAI Whisper (Faster-Whisper), Streamlit, NLTK, Plotly, Hugging Face Transformers

1. Introduction

Podcasts and long-form audio content have become one of the most popular mediums for sharing information, discussions, and knowledge across various domains such as education, healthcare, business, and entertainment. However, extracting meaningful insights from lengthy audio recordings is often time-consuming, as users must listen to entire episodes to locate specific topics or information.

To address this challenge, this project presents an AI-driven Podcast Intelligence System that automatically converts audio recordings into text, segments the content into coherent topics, and enriches each segment with summaries, keywords, and emotional context. By leveraging recent advancements in speech recognition and natural language processing, the system enables efficient navigation, search, and understanding of podcast content.

The proposed system integrates OpenAI Whisper for speech-to-text transcription, transformer-based NLP models for topic segmentation and summarization, and an interactive Streamlit user interface for visualization and exploration. This approach significantly reduces manual effort, improves content accessibility, and enhances user experience by allowing users to quickly identify and focus on relevant sections of an audio recording.

2. Problem Statement

Podcasts and long-duration audio recordings contain valuable information, but extracting specific topics or insights from them is difficult and time-consuming. Users are often required to listen to the entire audio to locate relevant discussions, which reduces efficiency and accessibility. Traditional transcription tools provide raw text output but lack topic segmentation, semantic organization, emotional context, and interactive navigation.

Moreover, existing systems do not effectively support searching within audio content, identifying topic boundaries, or understanding the sentiment and emotion of speakers across different segments. This limitation makes it challenging for users to quickly analyze, summarize, or retrieve meaningful information from large volumes of audio data.

Therefore, there is a need for an intelligent, automated system that can transcribe audio, segment it into meaningful topics, analyze sentiment and emotions, and provide an interactive interface for efficient navigation and exploration of podcast content.

3. Objectives of the Project

The primary objective of this project is to develop an AI-powered system that intelligently processes podcast and long-form audio content to extract meaningful insights and enable efficient navigation.

The specific objectives of the project are:

- To automatically convert podcast audio into accurate text using speech-to-text technology.
- To perform audio preprocessing for noise reduction and quality enhancement.
- To segment long transcripts into coherent topic-based sections using semantic analysis.
- To generate summaries, titles, and key points for each identified topic.
- To extract important keywords to improve content understanding and searchability.
- To analyze sentiment and emotions present in the audio at both topic and segment levels.
- To enable semantic search and indexing for quick retrieval of relevant content.
- To design an interactive and user-friendly interface for topic navigation and visualization.
- To provide an AI-based question–answering assistant for conversational exploration of audio content.

4. Dataset Selection: Audio_recording_whisper

For the development and validation of this system, we selected the Audio_recording_whisper dataset from Kaggle as our primary corpus. This dataset consists of realistic doctor–patient conversational audio recordings, making it highly suitable for evaluating speech recognition and conversational intelligence systems.

- **Conversational Diversity:**

The dataset captures natural doctor–patient interactions involving questions, explanations, pauses, and follow-ups. This diversity allows us to rigorously evaluate the Whisper model’s ability to handle spontaneous speech, varying speaking styles, and conversational dynamics.

- **Domain-Specific Complexity:**

Medical conversations include symptom descriptions, medical terminology, and contextual explanations. This semantic complexity effectively tests the performance of our summarization (DistilBART) and keyword extraction models in generating meaningful and concise insights.

- **Real-World Structure:**

The structured yet conversational flow of clinical dialogues provides a reliable foundation for validating our **topic segmentation logic**, as discussions naturally transition between symptoms, history, diagnosis, and lifestyle factors.

5. Scope of the Project

The scope of this project is focused on the design and implementation of an AI-based audio intelligence system that processes long-form conversational audio and transforms it into structured, searchable, and insightful information.

- **Audio Transcription:**

The system supports automated transcription of conversational audio recordings using OpenAI Whisper, converting speech into accurate, timestamped text.

- **Topic Segmentation and Summarization:**

The project includes semantic segmentation of transcripts into meaningful topics, along with the generation of summaries, titles, and key points for each segment.

- **Keyword and Emotion Analysis:**

The system extracts important keywords and performs sentiment and emotion analysis to provide contextual understanding of each topic and speaker segment.

- **Semantic Search and Navigation:**

Users can perform keyword-based and semantic searches to quickly locate relevant sections of the audio without listening to the entire recording.

- **Interactive Visualization:**

The scope covers the development of an interactive Streamlit-based interface that visualizes topic timelines, emotion flows, and allows audio playback at the topic level.

- **Limitations:**

The project is limited to offline audio processing, English-language content, and does not include real-time transcription or clinical diagnosis.

6. System Architecture

The system follows a modular, pipeline-based architecture:

Audio Input → Preprocessing (Cleaning & Noise Reduction) → ASR (OpenAI Whisper) → NLP Processing (Topic Segmentation & NLTK) → Intelligence Layer (Summarization, Keyword Extraction & Sentiment Analysis) → Frontend (Streamlit Dashboard)

7. Module Description

7.1 Audio Preprocessing Module

This module prepares raw audio files for accurate transcription. It utilizes LibROSA, SoundFile, and NumPy to perform audio resampling, mono conversion, noise reduction, and amplitude normalization. These preprocessing steps help reduce background noise and improve the overall quality of the audio signal, thereby enhancing transcription accuracy.

7.2 Transcription Module

The transcription module converts preprocessed audio into text using the OpenAI Whisper model (via Faster-Whisper), supporting multiple model sizes such as *tiny*, *base*, and *small*. The module generates timestamped speech segments, enabling precise alignment between the audio and textual content.

7.3 Topic Segmentation Module

This module identifies topic transitions within the transcript using semantic similarity-based segmentation. Sentence embeddings are generated using transformer models, and cosine similarity is applied across sliding windows to detect shifts in conversational context. Each resulting segment represents a meaningful and coherent discussion topic.

7.4 Intelligence Module (Keywords, Summaries, Sentiment)

- **Keywords:**

Important keywords are extracted using KeyBERT and YAKE, leveraging semantic embeddings and statistical methods to identify contextually relevant terms for each topic.

- **Summarization:**

The DistilBART Transformer model is used to generate concise summaries, titles, and bullet points for each topic segment, enabling quick understanding of long discussions.

- **Sentiment&Emotion:**

Sentiment polarity and emotional states are analyzed using DistilBERT and DistilRoBERTa emotion models, providing both topic-level sentiment and segment-level emotional context.

7.5 Frontend Visualization Module

The Streamlit-based frontend presents the processed results through an interactive dashboard that includes:

- Full, scrollable transcript with topic-wise organization
- Topic timeline visualization showing segment duration
- Emotion flow timeline illustrating sentiment and emotional intensity
- Semantic search with highlighted results and timestamp navigation
- Audio playback for individual topic segments

This module enables users to intuitively explore, analyze, and interact with long-form audio content.

8. Technology Stack

- **Programming Language:** Python
- **Speech-to-Text:** OpenAI Whisper (Faster-Whisper)
- **Audio Processing:** LibROSA, SoundFile, NumPy
- **Natural Language Processing:** NLTK, Hugging Face Transformers
- **Summarization Model:** DistilBART
- **Sentiment & Emotion Analysis:** DistilBERT, DistilRoBERTa
- **Keyword Extraction:** KeyBERT, YAKE
- **Semantic Embeddings:** Sentence-Transformers (MiniLM)
- **Visualization:** Plotly
- **User Interface:** Streamlit
- **Machine Learning Framework:** PyTorch

9. Project Structure

The project is organized in a modular and well-structured directory hierarchy to ensure clarity, maintainability, and scalability. Each directory is responsible for a specific stage of the audio processing and analysis pipeline.

```
project/
├── audio_raw/
├── audio_processed/
├── docs/
├── logs/
├── segments/
├── transcripts/
├── src/
│   ├── preprocessing.py
│   ├── transcription.py
│   ├── segmentation.py
│   ├── summarization.py
│   ├── keyword_extraction.py
│   ├── sentiment_analysis.py
│   ├── indexing.py
│   ├── chatbot.py
│   ├── logger.py
│   └── ui_app.py
├── test_processed/
├── tests/
│   ├── __init__.py
│   ├── test_chatbot.py
│   ├── test_detailed_emotion.py
│   ├── test_diarization_logic.py
│   ├── test_emotion_manual.py
│   ├── test_indexing.py
│   ├── test_keyword_extraction.py
│   ├── test_logger.py
│   ├── test_preprocessing.py
│   ├── test_segmentation.py
│   ├── test_sentiment.py
│   ├── test_summarization.py
│   └── test_transcription.py
├── LICENSE
├── README.md
└── requirements.txt
```


10. Implementation Journey (Week-by-Week)

Week 1: Project Setup & Feasibility

- Selected the `Audio_recording_whisper` dataset from Kaggle, consisting of doctor–patient conversational audio.
- Analyzed dataset characteristics such as audio length, speaker count, and conversational structure.
- Defined the project structure and configured dependencies using `requirements.txt`.

Week 2: Audio Preprocessing & Transcription Pipeline

- Implemented the audio preprocessing module to perform noise reduction, normalization, resampling, and mono conversion.
- Integrated OpenAI Whisper (Faster-Whisper) for speech-to-text transcription.
- Generated timestamped transcript segments for accurate alignment between audio and text.
- Result: Successfully converted conversational audio into structured text with timestamps.

Week 3: Topic Segmentation Logic

- Designed and implemented semantic topic segmentation using sentence embeddings and cosine similarity.
- Applied a sliding window approach to detect contextual shifts within the transcript.
- Topic boundaries were identified based on semantic similarity drops between consecutive segments.
- Outcome: Long conversations were segmented into coherent and meaningful discussion topics.

Week 4: Intelligence Layer (Summarization, Keywords & Sentiment)

- Integrated DistilBART to generate concise summaries, titles, and bullet points for each topic.
- Implemented keyword extraction using KeyBERT and YAKE for semantic relevance.

- Applied sentiment and emotion analysis using transformer-based models to capture emotional context at topic and segment levels.

Week 5: Full-Stack UI Integration

- Developed an interactive frontend using Streamlit.
- Integrated Plotly visualizations for topic timelines and emotion flow.
- Implemented semantic search and keyword highlighting, allowing users to jump directly to relevant timestamps.
- Enabled topic-wise audio playback and chatbot-based query interaction.

Week 6: System Optimization & Final Enhancements

- Improved performance using Streamlit caching (`@st.cache_resource`, `@st.cache_data`) to avoid repeated model loading.
- Added session-based logging for debugging and transparency.
- Conducted extensive unit testing for preprocessing, transcription, segmentation, sentiment, and chatbot modules.
- Finalized UI styling and ensured smooth end-to-end pipeline execution.

11. Future Enhancements

While the current system effectively performs automated transcription, topic segmentation, and intelligent analysis of conversational audio, several enhancements can be incorporated in future versions to improve scalability, usability, and performance.

- **Multilingual Support:**
Extend the system to support multiple languages by leveraging Whisper's multilingual capabilities, enabling analysis of non-English audio content.
- **Real-Time Transcription:**
Incorporate real-time or streaming transcription to process live audio inputs such as online meetings, interviews, or telemedicine sessions.

12. Conclusion

This project presents an AI-powered system designed to automatically transcribe and analyze long-form conversational audio content. By integrating OpenAI Whisper for speech-to-text transcription with advanced natural language processing techniques, the system effectively converts raw audio into structured text and segments it into meaningful topics. The inclusion of summarization, keyword extraction, and sentiment–emotion analysis enhances the interpretability of the content, allowing users to quickly understand key discussions without listening to the entire recording.

Furthermore, the interactive Streamlit-based interface enables efficient navigation through transcripts using semantic search, topic timelines, and audio playback. The modular pipeline architecture ensures scalability, maintainability, and ease of future enhancement. Overall, the system demonstrates the practical applicability of modern AI and NLP technologies for intelligent audio analysis and serves as a strong foundation for extending the solution to real-time processing, multilingual support, and large-scale deployment.